HOVER: History- and Output-Conditioned Variable-Shared Error Refinement for Frozen Time-Series Forecasters

Tian Gan
Shandong Technology and Business University, China
2025410065@sdtbu.edu.cn

Junshou Li
Shandong Technology and Business University, China
xor7xiaohei@gmail.com

Shiming Fan
Shandong Technology and Business University, China
2024410061@sdtbu.edu.cn

Fan Zhang (corresponding author)
Shandong Technology and Business University, China
zhangfan51@sina.com

***Abstract*—Trained multivariate forecasters are often reused as existing prediction components. Organizing recent history and a model's own forecast trajectory to learn instance-specific residual corrections is a modeling challenge when reusing these predictive capabilities. We propose History- and Output-Conditioned Variable-Shared Error Refinement (HOVER), which learns direct historical and forecast-conditioned residuals from a shared historical representation. The latter represents the base forecast trajectory relative to the latest observation and modulates historical features through a low-dimensional multiplicative interaction. Both residuals are restored to the local scale and added to the frozen forecast, with parameters shared across variables. Matched-backbone and multi-horizon DEV experiments show that refinement reduces mean MSE in the evaluated settings. Capacity-matched and structural controls support the two-path organization, while prespecified independent held-out tests further reveal that the gains vary by task. The results suggest that structuring history and existing forecasts together offers a viable approach to error refinement while preserving the original prediction interface.**

***Index Terms*—Multivariate time-series forecasting, frozen forecasters, forecast refinement, output conditioning, variable-shared modeling.**

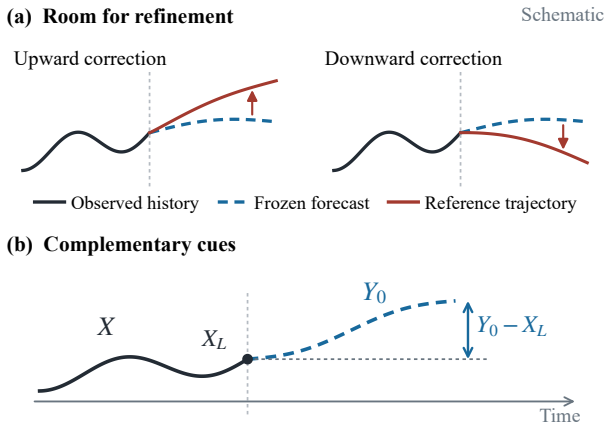


Fig. 1. Motivation for history- and forecast-conditioned refinement. (a) Frozen forecasts may retain residual errors with different directions and shapes. (b) Recent observations and the base forecast relative to the latest observation provide complementary representations for instance-specific residual correction. Curves are schematic.

I. Introduction

Multivariate time-series forecasts inform traffic operations [1], [2] and tasks such as electricity dispatch and energy planning. Forecasting architectures now range from attention-based models [3], [4] and linear predictors [5] to patch-based [6] and multiscale designs [7]. These models are often trained for a particular data source and then reused. This makes improvement of an existing forecast, without retraining its underlying predictor, a valuable capability in its own right.

A trained forecaster already fits a data pipeline with established variables, preprocessing, time inputs, and a forecast horizon. Keeping that mapping fixed makes it possible to improve the output while retaining these dependencies. It also gives us a concrete reference: we can compare the corrected forecast with the original forecast on exactly the same target.

Figure 1 illustrates why the correction can vary across windows and how recent history and the predicted trajectory represent complementary cues. Studies of non-stationary time series and instance normalization highlight the role of local state and scale [8], [9]. Historical observations describe what has just happened; the forecast expresses how the trained model expects the series to evolve. These two representations can guide a correction that is specific to the present forecasting instance. For a deterministic predictor, the forecast is a function of its inputs; making it explicit provides a structured representation of anticipated evolution, rather than an independent observation of the future.

Existing methods already refine forecasts after training. The output-side formulation of δ-Adapter [10] can condition on both history and the base prediction. COSA [11] updates an adapter as observations become available, and UEC-STD [12] addresses error accumulation during autoregressive forecasting. For a complete multi-step forecast corrected offline, the question we study is how to organize the two representations into a residual for the current instance.

A residual can respond directly to the recent state. It can also depend on how the backbone extrapolates that state: similar

histories can call for different adjustments when the forecast trajectories differ. We give these responses separate output routes over a common historical representation. Sharing this representation and the associated maps across variables lets each channel supply its own state and forecast while using the same correction rule.

The resulting design raises two questions about the residual routes: whether conditioning on the current forecast improves a direct historical correction, and whether a direct historical output remains useful once the conditioned path is present. The interaction and the choice of conditioning signal can then be tested while retaining the two-path framework.

These questions concern how a fixed amount of input information is used. The trajectory exposes a prediction already learned by the backbone, and sharing a historical representation gives the two residual heads a common starting point. A capacity-matched control examines whether this organization matters when a simpler corrector has the same inputs and a similar parameter count.

Time-series data engineering has studied reusable representations [13], [14], continual forecasting over streams [15], and automated predictor selection [16]. We address the stage after a predictor is chosen and trained. An output refinement module can be fitted separately while the backbone keeps its native preprocessing and forecasting interface.

We propose HOVER (History- and Output-Conditioned Variable-Shared Error Refinement). A shared encoder extracts features from each normalized historical window. The history residual path (HRP) directly maps these features to a correction; the forecast-conditioned refinement path (FCRP) encodes the base prediction relative to the latest observation and combines the predicted trajectory with historical features through low-dimensional multiplicative interaction. Both corrections are restored to the local historical scale before being added to the frozen forecast.

The two paths have distinct roles but share the same historical representation. HRP responds directly to the observed state, whereas FCRP adjusts its output to the shape of the current forecast. The encoder, projections, and residual heads are shared across variables. For fixed input and prediction lengths, the additional parameter count therefore remains independent of the number of variables.

The experiments examine the role of this information organization. Complete-system comparisons assess final accuracy, matched backbones measure refinement gains, and capacity-matched and structural controls examine design choices. Horizon experiments, preselected held-out tests, and actual forecast trajectories provide complementary views of performance and its scope.

These comparisons also distinguish two objectives that need not favor the same backbone. A predictor with larger residual error may permit a greater relative reduction, whereas a small reduction on an accurate predictor may yield the lowest final error. We therefore use NLinear and OLinear as complementary fixed instances of one generic method. The former exposes a substantial correction range and supports the mechanism study; the latter tests refinement on a competitive native predictor and across horizons. Neither instance selects a different backbone for each dataset. The contributions are:

- We introduce a variable-shared, two-path refinement module for frozen multivariate forecasters, learning direct historical and forecast-conditioned corrections from a shared historical representation.
- We encode the base prediction relative to the latest observation and use low-dimensional multiplicative interaction to condition residual refinement on the predicted trajectory.
- We establish the method's predictive value through complete-system comparisons, matched cross-backbone evaluation, parameter-matched controls, twelve-task ablations, and adaptation at multiple prediction horizons.

II. Related Work

A. Multivariate Time-Series Forecasting

Long-range forecasting can be learned through attention-based temporal models such as Informer, Autoformer, and FEDformer [3], [4], [17], or through compact mappings such as DLinear, NLinear, FITS, and SparseTSF [5], [19], [20]. These alternatives matter for refinement because their outputs are shaped by different modeling assumptions. Evaluating a common corrector across them tests whether its design depends on one particular predictive mapping.

Forecasting architectures organize time and variables through patches, variable tokens, cross-dimension or periodic representations [6], [21], [22], [23], as well as frequency, convolutional, and multiscale processing [7], [24], [25], [26], [27]. Other designs incorporate exogenous variables or transformed domains [28], [29]. These differences make multiple frozen backbones a meaningful setting for evaluating a common output-refinement interface.

An output interface nevertheless does not erase the distinctions between these models. A backbone can jointly process variables or operate channel-wise, use calendar inputs or only observations, and represent temporal structure in different domains. A compatible corrector must preserve these choices when obtaining the base forecast. In particular, sharing adapter parameters across variables is different from learning cross-variable interactions inside the adapter. Any such interactions already used by a backbone can affect the correction through its output, without requiring the refinement module to reproduce its internal representation.

Time-series research also uses graph structure and historical patterns [18], [30], [31], and studies model selection, auxiliary information, and distillation [16], [32], [33]. Autoregressive models have a different error-propagation process [34], while pretrained forecasters expand the range of models available for reuse [35], [36], [37], [38], [39]. We examine output refinement after a predictor has been selected and trained.

B. Frozen-Forecaster Adaptation and Error Correction

δ-Adapter [10] studies input nudges and bounded output adjustments around a frozen forecast. Its additive output formulation can already combine history and the base prediction.

HOVER instead specifies how to organize the residual: one head responds directly to shared historical features, and a second uses a relative forecast and multiplicative interaction to condition the correction. Our capacity and structure controls examine this organization. The direct comparison uses the official offline multiplicative PostY variant.

COSA [11] conditions a gated output correction on the base prediction and recent observation statistics, updating its adapter as outcomes become available. UEC-STD [12] separates trend and seasonal errors that accumulate during autoregressive rollout. The setting here is a complete direct forecast, with an offline-fitted correction and fixed weights during evaluation. This setup makes it possible to study the residual structure under a common prediction and feedback protocol.

Local statistics also offer a useful representation of the current window. RevIN [8] normalizes individual instances, while Non-stationary Transformers [9] study how to preserve informative dynamics under stationarization. HOVER uses window-specific statistics to encode history and restore the predicted residual, while retaining each frozen forecaster's original input processing.

C. Combining History with Forecast Outputs

Historical observations describe the recent state; a forecast expresses the model's estimated future evolution. Other forecasting approaches incorporate exogenous inputs [28], historical patterns [31], or auxiliary signals [32]. HOVER uses the *current base forecast* as a conditioning signal: its trajectory relative to the last observation modulates a correction derived from shared historical features. The module thus conditions its residual on both observed dynamics and the forecast being refined.

This representation choice is also distinct from introducing exogenous measurements. The forecast is produced from the predictor's existing legal inputs. Its utility to a finite refinement network can arise from exposing a learned future trajectory that would otherwise have to be reconstructed from history. Our experiments test task-specific adaptation of this representation across several backbone families; they do not establish zero-shot transfer of one adapter among the foundation models discussed above.

III. Method

A. Problem Setup and Overview

Consider a historical window $X \in \mathbb{R}^{L\times C}$ with C variables and L time steps. The forecasting objective is to predict the next H time steps, $Y \in \mathbb{R}^{H\times C}$. Given a trained base forecaster F_ϕ, HOVER learns a separate residual module conditioned on the observed history and the base output. The backbone parameters ϕ remain frozen during refinement training:

$$Y_0 = F_\phi(X), \qquad \widehat{Y} = Y_0 + R_\theta(X, Y_0). \tag{1}$$

Forecasters that use time features retain those available at prediction time.

The variables X, Y, and Y_0 in these equations use the same dataset-standardized coordinates. Dataset-level statistics are fitted on training observations and remain fixed. The local normalization introduced below is an internal coordinate system for the adapter, not a replacement for the backbone's preprocessing. In batched computation the input has shape $B\times L\times C$ and both forecasts have shape $B\times H\times C$. The adapter processes each sample–variable pair with shared mappings, while keeping that pair's local statistics separate.

The module comprises a variable-shared historical encoder, a history residual path (HRP), and a forecast-conditioned refinement path (FCRP). The encoder represents the normalized history of each variable by a feature h. HRP directly maps this feature to a forecast-horizon correction, whereas FCRP combines h with the base forecast's change relative to the latest observation. The two residuals are restored to the historical scale and added to Y_0, as illustrated in Fig. 2.

B. Shared Historical Encoding and Direct Correction

We first transpose the history into a variable-wise representation $Z = X^\top \in \mathbb{R}^{C\times L}$, where Z_c is the L-step history of variable c. We compute a local mean μ_c, scale s_c, and normalized history $\widetilde{Z}_c$ for every sample and variable:

$$\begin{aligned} \mu_c &= \frac{1}{L}\sum_{t=1}^{L} Z_{c,t}, \\ s_c &= \sqrt{\frac{1}{L}\sum_{t=1}^{L}(Z_{c,t}-\mu_c)^2 + \epsilon}, \\ \widetilde{Z}_c &= (Z_c - \mu_c)/s_c. \end{aligned} \tag{2}$$

Here ϵ ensures numerical stability. These window-specific statistics are distinct from dataset-level standardization.

We use population variance and $\epsilon = 10^{-5}$. Only the observed L steps enter these statistics. This is important when a variable is nearly constant: the stabilizer keeps the coordinate transform defined without consulting the future. Since the predicted quantity is a residual, restoring it later requires multiplication by s_c but no addition of μ_c. Adding the historical mean again would introduce a level term already present in the base forecast. A variable-shared encoder E_h, consisting of two linear layers and GELU, produces historical feature h_c. The HRP head G_h predicts a residual for the entire future horizon:

$$h_c = E_h(\widetilde{Z}_c), \qquad r_c^h = G_h(h_c). \tag{3}$$

The same encoder and output head are applied to every variable. The historical feature h_c is also passed to FCRP, giving the two branches a shared representation of the observed state.

C. Forecast-Shape Encoding and Conditional Refinement

FCRP additionally uses the trajectory predicted by the frozen backbone. For variable c, we anchor the base forecast $Y_{0,c}$ at the most recent observed value $X_{L,c}$ and normalize the resulting change by the historical scale:

$$d_c = \frac{Y_{0,c} - X_{L,c}\mathbf{1}_H}{s_c}, \qquad a_c = E_f(d_c), \tag{4}$$

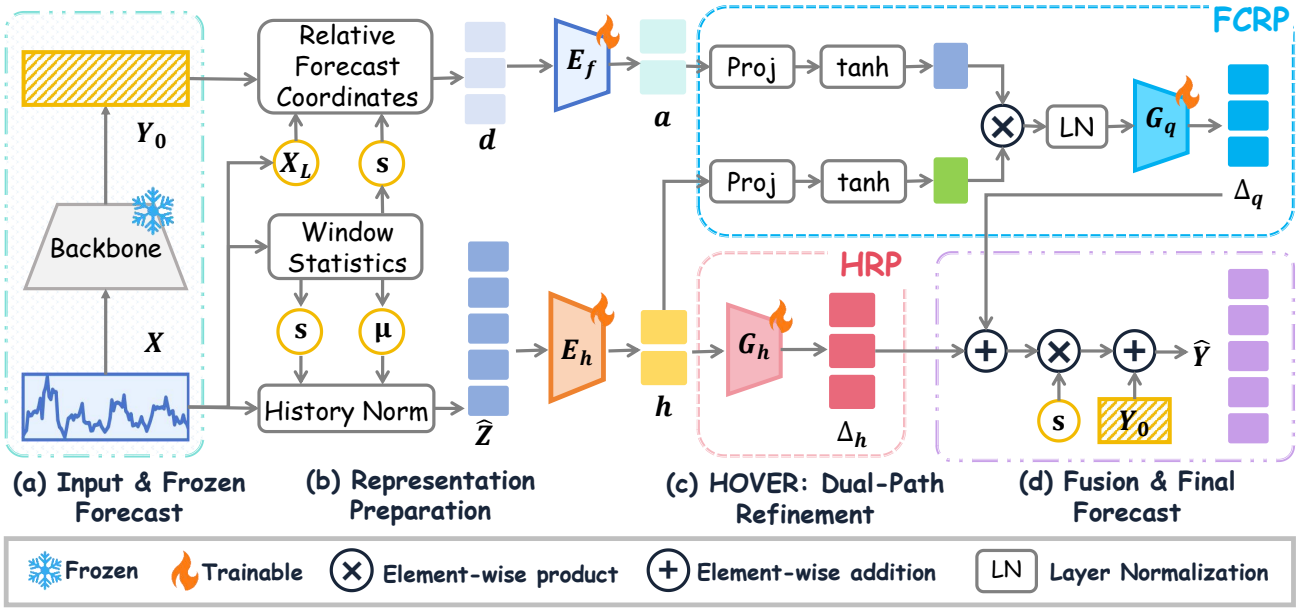


Fig. 2. Architecture of HOVER. The frozen backbone produces a base forecast. Historical observations are normalized per variable, and the base forecast is represented relative to the latest observation. HRP and FCRP produce a direct historical residual and a forecast-conditioned residual, respectively. Their sum is rescaled and added to the base prediction. Snowflake and flame symbols mark frozen and trainable components. The figure's Δ_h and Δ_q denote the normalized residuals r^h and r^q.

where $\mathbf{1}_H$ is the H-dimensional all-ones vector. Forecast encoder E_f comprises two linear layers and GELU. The representation a_c encodes the direction and magnitude of change relative to the present state.

The last observation and the historical mean serve different purposes. Mean centering describes variation within the historical window, whereas the last-value anchor places the future trajectory relative to the point from which forecasting begins. The H coordinates retain the predicted changes at individual future positions; they are not reduced to a single level shift or slope. Both history and forecast changes are expressed in the same local scale, so the interaction can respond to their relative shapes without using the unknown future target.

To let the forecast modulate individual historical features, we first project both representations into a common low-dimensional space:

$$u_c = W_h h_c + b_h, \qquad v_c = W_f a_c. \tag{5}$$

A nonlinear element-wise product combines the two projected features, followed by layer normalization and a forecast-horizon output head G_q:

$$q_c = \mathrm{LN}[\tanh(u_c) \odot \tanh(v_c)], \quad r_c^q = G_q(q_c). \tag{6}$$

Here $\odot$ denotes element-wise multiplication. The forecast feature modulates the history-dependent correction so that the residual responds to the specific base prediction. All encoding, projection, and output parameters are shared across variables.

In the fixed implementation, E_h uses two 64-dimensional hidden layers, and E_f uses two 32-dimensional layers; each linear encoder layer is followed by GELU. The projections and layer normalization have dimension 32. The product in Eq. (6) allows forecast features to modulate the response to historical features coordinate by coordinate. Its 32-coordinate interaction supplies a compact mechanism for conditioning the residual. HRP provides a direct route from history to the output, so the model need not express every correction through this multiplicative interaction.

D. Two-Path Fusion and Scale Restoration

HRP and FCRP predict r_c^h and r_c^q on the normalized scale. Their sum is restored using the local scale s_c and added to the frozen prediction:

$$\widehat{Y}_c = Y_{0,c} + s_c\big(r_c^h + r_c^q\big). \tag{7}$$

Both residual output heads, G_h and G_q, are zero-initialized. The refined model therefore begins with exactly the frozen backbone's prediction:

$$\widehat{Y}_{\text{initial}} = Y_0. \tag{8}$$

Because the encoders, interaction layers, and residual heads operate channel-wise with shared parameters, the adapter parameter count does not grow with the number of variables for fixed L and H. Forecast encoders and output mappings are sized for the prediction horizon in each experiment.

Counting the linear weights and biases and the affine parameters of layer normalization gives

$$P_{\text{HOVER}}(L, H) = 64L + 130H + 8480. \tag{9}$$

At $L = 336$ and $H = 96$, this is 42,464 additional trainable parameters. The absence of C from this expression describes

parameter sharing, not constant computational cost: applying the maps still requires computation for every variable. A change in H changes the forecast encoder input and both residual output heads. Each horizon therefore uses a separately fitted, dimension-matched adapter; a 96-step adapter is not reused as a longer-horizon output head.

The zero-initialized heads make the existing prediction the starting point for learning. As their weights move away from zero, the heads pass gradients back to the encoders, allowing the historical and forecast-conditioned representations to adapt to the supervised residual objective.

a) Behavior of the refinement coordinates.: The local coordinates have a direct interpretation under a change of level. Suppose a constant is added to all historical values and all base-forecast values of one variable. Its historical mean changes by the same constant, its scale is unchanged, and both the centered history and the relative forecast remain unchanged. Consequently, the adapter produces the same residual, and the final forecast follows the new level through the base prediction. This property concerns the refinement interface with both inputs shifted together; the native predictor retains its own response to a changed input. It explains why the adapter can organize corrections around the present trajectory rather than allocate separate parameters to each absolute level.

Variable sharing provides a related consistency property. Reordering the paired history and forecast channels reorders their local statistics, features, and residual outputs in the same way, because every channel is processed by the same maps. The resulting correction follows the channel order. At the same time, two channels can receive different corrections because their histories and base forecasts differ. Sharing therefore ties the correction rule while preserving instance-specific outputs. This is useful for multivariate data with many channels: a common rule is learned from their training examples, and each evaluation window supplies its own local state and forecast shape.

With the history held fixed, the relative forecast can change the projected product and hence the conditioned residual. The direct head still gives the model a way to respond to the observed state. This separation motivates the two route-removal controls: each tests whether the other route can provide a comparable correction after retraining.

E. Training and Inference

We freeze both the parameters and inference state of the native forecaster and optimize only the new encoders, projections, and residual heads. Training minimizes the final forecast error together with a regularizer that retains proximity to the base prediction:

$$\mathcal{L} = \mathrm{MSE}(\widehat{Y}, Y) + \lambda\,\mathrm{MSE}(\widehat{Y}, Y_0). \tag{10}$$

The coefficient λ is selected on the prescribed inner-validation split. At inference time, the frozen backbone first produces Y_0, after which HOVER computes both corrections and restores their scale. Each backbone retains its original time-feature interface; adapters are trained separately against their corresponding frozen checkpoints.

During training, the target enters only Eq. (10); gradients update θ alone. Both training and evaluation obtain Y_0 from the backbone in evaluation mode.

F. Interpreting the Residual Objective

The objective can be understood directly in terms of the error left by the frozen predictor. Write $e = Y - Y_0$ and $g = R_\theta(X, Y_0)$. Apart from the common normalization by the number of target elements, Eq. (10) minimizes

$$\mathbb{E}\big[\|e - g\|_F^2 + \lambda\|g\|_F^2\big]. \tag{11}$$

For square-integrable targets, $\lambda \geq 0$, and an unrestricted residual function, let $\mathcal{I} = (X, Y_0)$ and $m(\mathcal{I}) = \mathbb{E}[e \mid \mathcal{I}]$. Completing the square in the conditional risk gives

$$\begin{aligned}&\mathbb{E}[\|e - g\|_F^2 + \lambda\|g\|_F^2 \mid \mathcal{I}] \\ &\quad = (1+\lambda)\left\|g - \frac{m(\mathcal{I})}{1+\lambda}\right\|_F^2 + K(\mathcal{I}),\end{aligned} \tag{12}$$

where $K(\mathcal{I})$ is independent of g. The ideal conditional residual is consequently

$$g^*(\mathcal{I}) = \frac{1}{1+\lambda}\mathbb{E}[Y - Y_0 \mid \mathcal{I}]. \tag{13}$$

This elementary calculation explains two aspects of the existing loss. The learnable target is the part of the residual predictable from the available inputs, and the retention term shrinks that conditional correction toward zero. Equation (13) interprets the objective in an unrestricted function space. HOVER approximates this conditional residual with its finite shared representation and supervised training.

The regularizer operates on the restored output, not on an arbitrary hidden-feature norm. For a given variable and window, $g_c = s_c(r_c^h + r_c^q)$; therefore its contribution is proportional to $s_c^2\|r_c^h + r_c^q\|^2$. The same normalized correction can have a different cost at a different local scale. Moreover, the penalty constrains the combined residual rather than separately forcing each branch toward a unique solution. These details align the objective with the final forecast coordinates and help explain why a branchwise interpretation requires more than the total training loss.

For a deterministic frozen mapping $Y_0 = F_\phi(X)$, conditioning additionally on Y_0 does not change the information available given the complete X: $\mathbb{E}[e \mid X, Y_0] = \mathbb{E}[e \mid X]$. If the backbone uses known time features, X in this statement denotes its complete legal input. FCRP's motivation is therefore representational. It exposes the future trajectory already constructed by the trained backbone and combines it with history through a specific inductive bias. The ablations examine whether this organization helps a finite network on the evaluated tasks, rather than testing whether a deterministic output creates new information.

IV. Experiments and Results

A. Tasks and Evaluation Protocol

We evaluate twelve multivariate forecasting datasets: ETTh1, ETTh2, ETTm1, ETTm2, ECL, Solar, Traffic, Exchange, PEMS03, PEMS04, PEMS07, and PEMS08. The benchmark studies [3], [40] provide context for the task design. In the main experiments, the history length is $L = 336$ and the forecast horizon is $H = 96$. The chronological roles used here are defined below.

We apply fixed training and model-selection protocols and evaluate each system with five random seeds. The primary metric is mean squared error (MSE), accompanied by mean absolute error (MAE). The main experiments report both metrics on DEV. The separate held-out evaluation reports TEST results. Paired evaluations use the same backbone checkpoint and matching seeds. The accompanying reproducibility materials contain per-seed results, standard deviations, and the configurations needed to reproduce the reported comparisons. Rankings and percentage gains are calculated from unrounded five-seed means, preserving the distinction between underlying comparisons and rounded table values.

a) Temporal roles and coordinates.: Windows follow chronological order. A DEV origin is the first future target position; its entire target interval lies within DEV, although its preceding history may include TRAIN observations. Dataset means and population standard deviations are computed from TRAIN only. Evaluation includes all forecast positions and variables; PEMS uses the flow channel and retains zero-valued targets. MSE and MAE are accumulated independently as squared and absolute errors divided by their respective valid target counts, in the accepted training-standardized coordinates.

The datasets differ in sampling interval and variable count, so a given prediction horizon corresponds to different elapsed times across tasks. Multi-horizon comparisons preserve each dataset's sampling interval. The repository documents temporal boundaries, channel identities, and window construction.

For hyperparameter selection, TRAIN is split chronologically into INNER_FIT (80%) and INNER_VAL (20%); normalization and OLinear's horizon-specific orthogonal transform are refitted within their respective training roles. Adapter hyperparameters are chosen within TRAIN. DEV is used both for adapter checkpoint selection and the reported development scores.

b) Training and selection.: Only the adapter is optimized after the backbone is frozen. Adapter training uses Adam and DEV-based early stopping, with learning rate and retention strength selected inside TRAIN before the five formal runs. Native predictors retain their established model-specific schedules. In particular, the paired NLinear and the native NLinear in Table I are separate training configurations. This distinction separates final-system accuracy from the effect of adding refinement to an otherwise unchanged frozen predictor. Complete configurations and results are available at https://github.com/GanTian1203/HOVER.

c) Paired aggregation.: For seed s, let E_s^A and E_s^B be the errors of an adapted system and its matched reference. We report $\Delta_s = E_s^A - E_s^B$, so a negative difference favors adaptation. A table gain is computed as

$$\mathrm{Gain}(A, B) = 100 \frac{\overline{E}^B - \overline{E}^A}{\overline{E}^B}, \tag{14}$$

The gain is the ratio of five-seed means, not the mean of seed-level percentages. Seed-level SD uses the sample denominator $5 - 1$, and winning-seed counts assess directional consistency. Each paired difference compares predictions on the same DEV targets.

B. Compared Systems and Experimental Questions

The complete-system comparison includes eight native predictors—NLinear, DLinear, FITS, PatchTST, iTransformer, SiGMA, TimeMixer, and OLinear—together with two refined systems, NLinear+HOVER and OLinear+HOVER. Each refined system uses its own frozen backbone and a separately trained adapter. For matched comparisons, we evaluate the adapter against the exact frozen checkpoint to which it was fitted. The paired NLinear used for refinement has a fixed training configuration distinct from the native NLinear entry in the full-system table.

We begin with the error attained by a complete forecasting system. Matching each refined system to its own frozen checkpoint then establishes how much the correction changes that error. With the predictive base held fixed, we can compare residual designs at similar capacity, examine their structural choices, and ask how the gains vary with the forecast horizon and evaluation period.

C. Accuracy of Complete Forecasting Systems

Adding a corrector is valuable if the resulting system remains competitive with independently trained forecasters. In Table I, OLinear+HOVER leads 15 of the 24 dataset–metric comparisons, with eight lowest mean MSEs and seven lowest mean MAEs. NLinear+HOVER leads three further comparisons, giving the two fixed refined systems 18 best results within this panel.

The strongest results arise from different predictor–corrector combinations. OLinear+HOVER leads both metrics on ETTm1, ECL, and all four PEMS tasks, and leads MSE on Solar and Traffic. NLinear+HOVER leads both metrics on ETTh2 and MSE on ETTm2. The two systems illustrate how refinement can build on different native predictors to achieve competitive absolute errors.

These ranks come from two fixed systems. DLinear leads both metrics on ETTh1, and PatchTST and native NLinear lead MSE and MAE, respectively, on Exchange. Native OLinear retains the lowest MAE on Solar and Traffic. Since the ranking reflects both the starting predictor and the correction, we next compare each refined system with its own starting point.

TABLE I
PERFORMANCE OF TEN FORECASTING SYSTEMS ON TWELVE DATASETS (FIVE-SEED DEV MEANS; LOWER IS BETTER).

Dataset	**HOVER (Ours)**				**Native forecasting systems**															
	OL+H		NL+H		OLinear		PatchTST		iTrans.		SiGMA		T.Mixer		NLinear†		DLinear		FITS	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	0.69503	0.55358	0.66005	0.54402	0.70877	0.56342	0.67281	0.55229	0.69312	0.56765	0.69206	0.55723	0.72111	0.57772	0.66423	0.54626	**0.64742**	**0.54123**	0.68139	0.56195
ETTh2	0.21527	0.31851	**0.20352**	**0.31118**	0.21811	0.32112	0.21099	0.31734	0.23161	0.33491	0.22334	0.32623	0.22183	0.32606	0.20530	0.31188	0.21714	0.32517	0.20944	0.31597
ETTm1	**0.35416**	**0.38880**	0.36098	0.39636	0.35435	0.38899	0.36618	0.40167	0.37200	0.40895	0.38624	0.41639	0.37686	0.40872	0.38591	0.40738	0.37875	0.40619	0.39060	0.41021
ETTm2	0.11574	**0.23142**	**0.11565**	0.23272	0.11636	0.23260	0.11807	0.23514	0.12939	0.25141	0.12195	0.23859	0.12125	0.23968	0.11602	0.23367	0.11737	0.23692	0.11651	0.23441
ECL	**0.10297**	**0.19155**	0.11434	0.20585	0.10340	0.19160	0.11485	0.20693	0.11153	0.20435	0.11285	0.20518	0.11233	0.20726	0.12036	0.21220	0.11951	0.21389	0.12537	0.22236
Solar	**0.14003**	0.18597	0.15505	0.20951	0.14328	**0.18122**	0.16071	0.23810	0.16053	0.22941	0.15072	0.21566	0.16131	0.22282	0.18761	0.23494	0.17539	0.24164	0.18844	0.23802
Traffic	**0.29868**	0.19653	0.32446	0.23167	0.30806	**0.19267**	0.32913	0.23618	0.31344	0.23211	0.32751	0.23438	0.33342	0.25287	0.33622	0.24096	0.33884	0.24448	0.33939	0.24641
Exchange	0.13475	0.25582	0.13003	0.24914	0.13322	0.25426	**0.12776**	0.25241	0.14576	0.27193	0.14349	0.26655	0.14742	0.26933	0.12792	**0.24786**	0.14540	0.26914	0.13587	0.26063
PEMS03	**0.05888**	**0.16017**	0.09874	0.21014	0.05941	0.16129	0.10266	0.21514	0.08226	0.18948	0.09390	0.20653	0.09723	0.21640	0.17471	0.27319	0.17091	0.28130	0.17606	0.27448
PEMS04	**0.08089**	**0.17820**	0.13173	0.23633	0.08193	0.18034	0.14034	0.24799	0.10349	0.21454	0.12931	0.23721	0.13878	0.25609	0.18631	0.27956	0.18045	0.28271	0.18808	0.28136
PEMS07	**0.09716**	**0.17893**	0.13900	0.23840	0.09776	0.18068	0.13507	0.23603	0.10103	0.20427	0.12367	0.22083	0.13624	0.24446	0.22466	0.30994	0.21656	0.31519	0.22704	0.31203
PEMS08	**0.19697**	**0.20433**	0.27024	0.25936	0.19783	0.20645	0.26282	0.26388	0.28933	0.27732	0.24036	0.24829	0.25843	0.25481	0.42945	0.34132	0.39591	0.33866	0.43672	0.35510
1st Count	**15**		**3**		**2**		**1**		**0**		**0**		**0**		**1**		**2**		**0**	

Red bold/blue underline: best/runner-up; *1st Count*: wins in 24 comparisons; †: different NLinear training configuration.

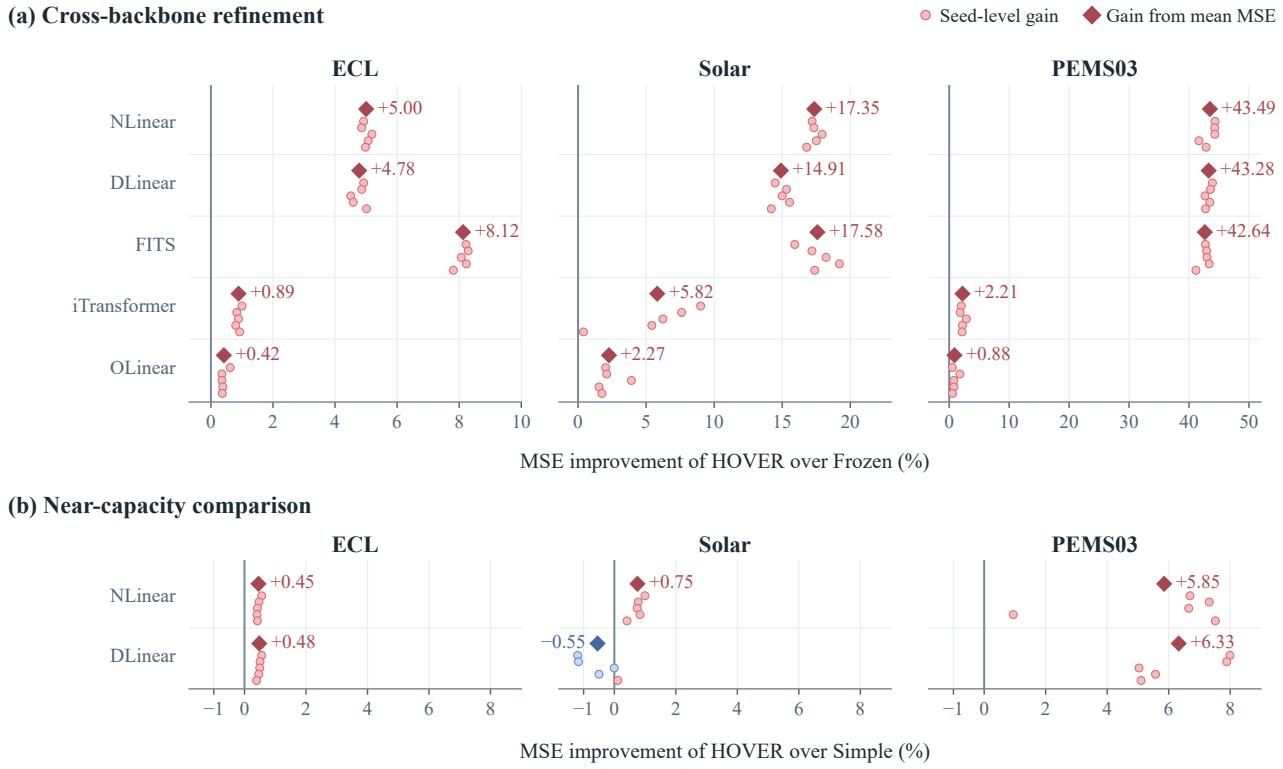


Fig. 3. Paired cross-backbone refinement and parameter-matched comparison. (a) MSE gains of HOVER over the frozen predictor on ECL, Solar, and PEMS03 for five backbones. Circles are seed-level paired gains; diamonds represent gains computed from the two five-seed mean MSE values. (b) Gains of HOVER over Simple for paired NLinear and DLinear. Each panel has a labeled linear horizontal scale, with zero indicating no MSE change. The paired NLinear differs in training configuration from the native NLinear in Table I.

D. Paired Gains on Frozen Backbones

Comparing forecasts from the same checkpoint on the same targets isolates the change introduced by the fitted corrector. Both NLinear+HOVER and OLinear+HOVER reduce mean MSE on eleven of the twelve datasets relative to their respective frozen backbones. The reduction for OLinear on Traffic, from 0.308064 to 0.298682, illustrates that a competitive starting forecast can still leave useful residual structure.

E. Cross-Backbone Adaptation

The paired comparison also asks whether the residual organization is useful across different forecasting architectures.

TABLE II
PAIRED REFINEMENT ACROSS FIVE FROZEN FORECASTERS AND THREE DATASETS (FIVE-SEED DEV MEANS).

Dataset	Backbone	Frozen		HOVER		Gain (%)	
		MSE	MAE	MSE	MAE	MSE	MAE
ECL	NLinear	0.12036	0.21220	0.11434	0.20585	+5.00	+2.99
	DLinear	0.11951	0.21389	0.11380	0.20789	+4.78	+2.80
	FITS	0.12537	0.22236	0.11518	0.20792	+8.12	+6.49
	iTrans.	0.11153	0.20435	0.11053	0.20332	+0.89	+0.51
	OLinear	0.10340	0.19160	0.10297	0.19155	+0.42	+0.02
Solar	NLinear	0.18761	0.23426	0.15505	0.20951	+17.35	+10.57
	DLinear	0.17539	0.24164	0.14924	0.21009	+14.91	+13.06
	FITS	0.18844	0.23802	0.15531	0.20866	+17.58	+12.34
	iTrans.	0.16053	0.22941	0.15119	0.21390	+5.82	+6.76
	OLinear	0.14328	0.18122	0.14003	0.18597	+2.27	-2.62
PEMS03	NLinear	0.17473	0.27358	0.09874	0.21014	+43.49	+23.19
	DLinear	0.17091	0.28130	0.09694	0.21277	+43.28	+24.36
	FITS	0.17606	0.27448	0.10098	0.21264	+42.64	+22.53
	iTrans.	0.08226	0.18948	0.08044	0.18702	+2.21	+1.30
	OLinear	0.05941	0.16129	0.05888	0.16017	+0.88	+0.69

We fit a separate HOVER corrector to each frozen NLinear, DLinear, FITS, iTransformer, and OLinear on ECL, Solar, and PEMS03. All fifteen pairs improve mean MSE, as do all 75 matched seed comparisons, and fourteen pairs improve mean MAE. This supports reuse of the architecture across linear, frequency-domain, and Transformer predictors.

The paired NLinear on PEMS03 illustrates the headroom for correction, with a 43.49% reduction in mean MSE. OLinear starts at a lower native MSE on this panel and attains the lowest adapted MSE on each of the three datasets. The paired errors and seed-level gains in Table II and Fig. 3(a) therefore describe complementary aspects of refinement: the reduction from a given starting point and the accuracy reached after correction.

On Solar, OLinear's lower MSE after refinement comes with higher MAE. The benefit thus depends on the error measure as well as the predictive base. To examine what the residual design contributes under these conditions, we next compare it with a correction that sees the same inputs and has a similar parameter budget.

F. Parameter-Matched Structural Comparison

To isolate the value of the two-path design, we compare HOVER against a shared-MLP adapter (Simple) of similar capacity. The comparison uses paired NLinear and DLinear backbones on ECL, Solar, and PEMS03, with Frozen, Simple, and HOVER sharing the same checkpoints, seeds, and input information. Simple maps historical and relative-forecast features into a single residual; HOVER instead combines the HRP and FCRP residuals. With $L = 336$ and $H = 96$, Simple has 42,096 trainable parameters and HOVER has 42,464.

With the available inputs and parameter counts closely matched, HOVER improves on the frozen forecast in all six comparisons and on Simple in five. On PEMS03, the MSE reductions relative to Simple reach 5.85% with NLinear and 6.33% with DLinear. In these settings, how the corrector organizes its inputs matters alongside the capacity available to it.

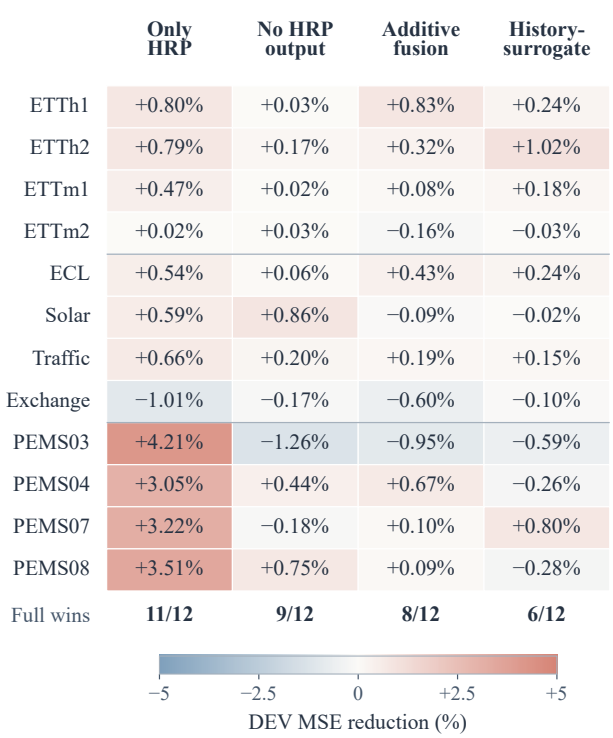


Fig. 4. MSE comparison between full HOVER and four retrained structural controls on twelve datasets. Each cell reports the relative reduction in five-seed mean DEV MSE achieved by full HOVER over the corresponding control. Positive values favor full HOVER, and negative values favor the control. Column summaries report the number of datasets favoring full HOVER.

Simple uses a two-layer shared MLP with a zero-initialized head on concatenated history and relative forecasts. Its lower MSE and MAE on DLinear–Solar (Table III and Fig. 3(b)) qualify the advantage of the two-path design. The comparison motivates changing one structural choice at a time to see when each part of the design is useful.

G. Structural Ablation of the Two Paths

We analyze the two-path structure on the paired frozen NLinear across twelve datasets. *Only HRP* uses the direct history residual alone. *No HRP output* removes that residual but keeps the shared historical encoder required by FCRP. *Additive fusion* replaces multiplicative feature interaction with projected additive fusion, and *History-surrogate* replaces the forecast-shape condition with an observed-history surrogate. All controls use matched data partitions, seeds, and frozen checkpoints.

The interventions answer different questions. Only HRP removes the forecast-conditioned route entirely; No HRP output removes only the direct output while preserving the historical features on which FCRP depends. Additive fusion specifically computes $\mathrm{LN}(\mathrm{GELU}(u_c + v_c))$ instead of Eq. (6); it is not feature concatenation. The surrogate uses the final

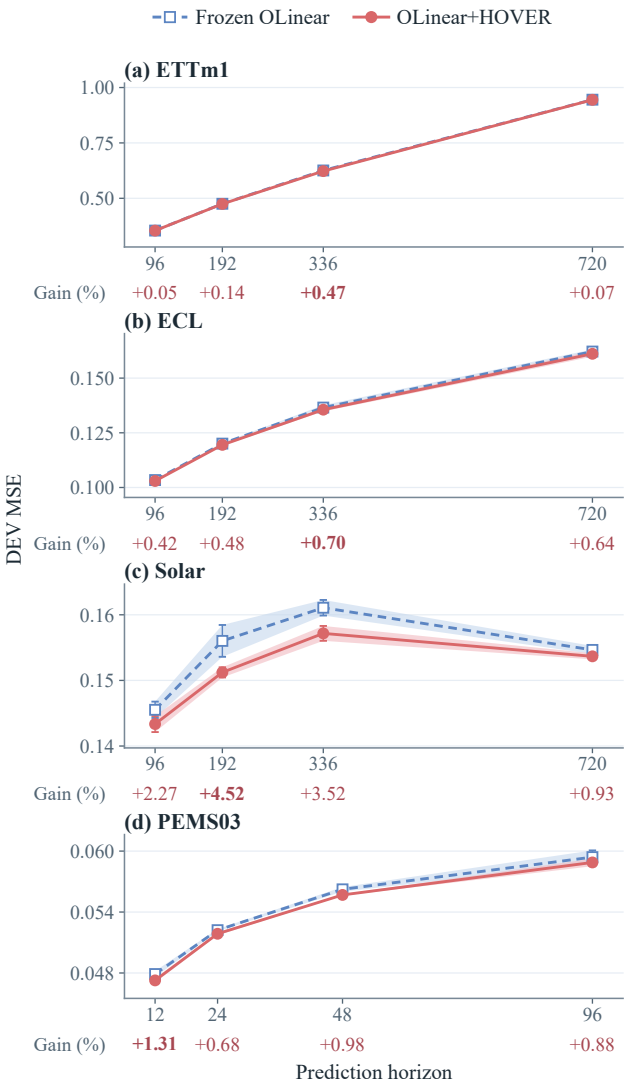


Fig. 5. Refinement of frozen OLinear across prediction lengths on ETTm1, ECL, Solar, and PEMS03. Curves show five-seed mean DEV MSE; error bars and bands indicate between-seed dispersion. Gain labels are computed from paired means at the corresponding horizon.

96 observed history steps, expressed relative to the same last observation and local scale, in place of the forecast coordinates. It never uses future labels. Each variant is trained with its own changed computation, so its error difference is a retrained structural comparison, not the numerical contribution obtained by switching off a branch after full-model training.

The comparisons in Fig. 4 and Table IV distinguish the two routes. Full HOVER lowers mean MSE relative to Only HRP in eleven tasks, including all four PEMS datasets with an equal-weight mean gain of 3.50%. This pattern supports conditioning the correction on the forecast trajectory alongside history. With the conditioned path already present, retaining a direct historical output lowers mean MSE in nine tasks. The two comparisons support learning both responses on a common historical representation.

TABLE III
CAPACITY-MATCHED PAIRED CONTROLS; GAINS ARE RELATIVE TO SIMPLE (FIVE-SEED DEV MEANS).

Dataset	Backbone	Frozen		Simple		HOVER		Gain (%)	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ECL	NLinear	0.12036	0.21220	0.11486	0.20645	0.11434	0.20585	+0.45	+0.29
	DLinear	0.11951	0.21389	0.11435	0.20845	0.11380	0.20789	+0.48	+0.27
Solar	NLinear	0.18761	0.23426	0.15622	0.21068	0.15505	0.20951	+0.75	+0.56
	DLinear	0.17539	0.24164	0.14843	0.20829	0.14924	0.21009	-0.55	-0.86
PEMS03	NLinear	0.17473	0.27358	0.10487	0.21647	0.09874	0.21014	+5.85	+2.93
	DLinear	0.17091	0.28130	0.10349	0.21875	0.09694	0.21277	+6.33	+2.73

The conditioned path raises a further question about the interaction and the conditioning input. Full HOVER lowers MSE relative to additive fusion in eight tasks. Replacing the relative forecast with observed history produces a closer MSE comparison, with six wins for the full design, while the full design lowers MAE in ten tasks. Exchange favors Only HRP, and removing the direct output lowers mean MSE on Exchange, PEMS03, and PEMS07. Together, these retrained comparisons show when a trajectory-conditioned, multiplicative design is useful and when a simpler route is competitive.

H. Adaptation Across Forecast Horizons

We test refinement of frozen OLinear over sixteen dataset–horizon settings. With $L = 336$, ETTm1, ECL, and Solar use $H \in \{96, 192, 336, 720\}$, while PEMS03 uses $H \in \{12, 24, 48, 96\}$. Each horizon is evaluated with its own frozen predictor and a horizon-matched adapter using paired seeds. HOVER lowers five-seed mean MSE in all sixteen settings and mean MAE in eleven (Table V and Fig. 5).

Changing the horizon changes both the target and the trajectory available to the corrector. The paired results show modest reductions across all four ECL horizons (0.42%–0.70%) and larger, more variable gains on Solar. The forecast-relative representation thus remains useful when the horizon changes and the adapter is fitted to the corresponding predictive base.

Lower mean MSE across horizons is also reflected in the matched runs: HOVER improves 73 of 80 seed pairs, each using the same frozen predictor and target positions within its horizon.

MAE rises on Solar at all four horizons and on ETTm1 at $H = 192$. Squared and absolute losses weight deviations differently, and the two metrics can move in different directions.

I. Representative Prediction Trajectories

Task-level errors pool windows and channels, leaving the shape and size of individual corrections implicit. Figure 6 shows two DEV cases, ECL with frozen OLinear and PEMS03 with frozen NLinear, matching the seed, channel, and origin within each case. The last 48 observed steps and a 24-step close-up of the 96-step forecast connect the learned change to the local trajectory.

The ECL correction changes the frozen curve only slightly, reducing window MSE from 0.15046 to 0.15001 (0.30%). On

TABLE IV
TWELVE-TASK ABLATION OF THE PAIRED FROZEN NLINEAR (FIVE-SEED DEV MEANS).

Task	Full HOVER		Only HRP		No HRP		Additive		History-surrogate	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	0.66005	0.54402	0.66539	0.54644	0.66023	0.54394	0.66561	0.54651	0.66164	0.54528
ETTh2	0.20352	0.31118	0.20514	0.31237	0.20387	0.31138	0.20418	0.31180	0.20562	0.31228
ETTm1	0.36098	0.39636	0.36270	0.39735	0.36105	0.39632	0.36128	0.39640	0.36162	0.39670
ETTm2	0.11565	0.23272	0.11568	0.23268	0.11569	0.23287	0.11547	0.23246	0.11562	0.23259
ECL	0.11434	0.20585	0.11497	0.20660	0.11440	0.20589	0.11483	0.20642	0.11462	0.20617
Solar	0.15505	0.20951	0.15597	0.21128	0.15640	0.21143	0.15491	0.20989	0.15501	0.20994
Traffic	0.32446	0.23167	0.32663	0.23388	0.32513	0.23222	0.32508	0.23261	0.32495	0.23227
Exchange	0.13003	0.24914	0.12873	0.24814	0.12981	0.24892	0.12926	0.24870	0.12990	0.24973
PEMS03	0.09874	0.21014	0.10308	0.21481	0.09751	0.20901	0.09780	0.20900	0.09815	0.20972
PEMS04	0.13173	0.23633	0.13588	0.24019	0.13232	0.23753	0.13262	0.23690	0.13139	0.23678
PEMS07	0.13900	0.23840	0.14363	0.24300	0.13874	0.23817	0.13913	0.23799	0.14012	0.23860
PEMS08	0.27024	0.25936	0.28006	0.26689	0.27227	0.26188	0.27049	0.25999	0.26949	0.25986
Full wins MSE	–		11/12		9/12		8/12		6/12	
Full wins MAE	–		10/12		7/12		8/12		10/12	

PEMS03, the change is more visible, with MSE falling from 0.05140 to 0.04106 (20.11%). These windows illustrate the different magnitudes of adjustment that the same residual design can produce when fitted to different forecasting tasks.

The DEV cases are chosen by gain rank and use a positively improving channel near the eligible-channel median. To assess how alternative correctors behave on the same input trajectory, we next hold the DLinear checkpoint and window fixed.

J. Local Behavior under Matched Refinement

Figure 7 puts Frozen, Simple, offline multiplicative PostY, and HOVER on the same DLinear forecasts. Over all DEV channels and origins, HOVER lowers mean MSE relative to Simple by 0.48% on ECL and 6.33% on PEMS03. The four fixed 24-step segments show reductions of 0.28%–0.70% and 3.96%–7.35%, respectively. The corresponding five-seed mean fractions of improved channel–origin windows, 51.6% and 56.7%, show how the aggregate gains coexist with windows that favor Simple.

Typical cases use frozen error and target shape at seed 951. Illustrative cases use the median gain among eligible windows where HOVER beats all three controls. The late decline in panel (c) is closer to the Simple forecast, which has lower window MSE. In panel (d), HOVER reduces error around the early rise, while its full-window MSE is lower than Simple's in two of five seeds. The two panels locate where the corrections differ in time and show why a favorable local change and the aggregate result answer different questions.

K. Chronological Held-Out Evaluation

We evaluate the preselected OLinear and paired NLinear checkpoints with and without HOVER on the chronological TEST portions of ECL, Solar, PEMS03, and Exchange. The four datasets, configurations, and five seeds (951–955) were fixed before TEST evaluation. The frozen pairs use the same TRAIN statistics, forecast origins, and complete 96-step targets. TEST serves only for inference and scoring after training and checkpoint selection.

The later segments test how far the development gains carry forward under the fixed configurations. Table VI shows lower

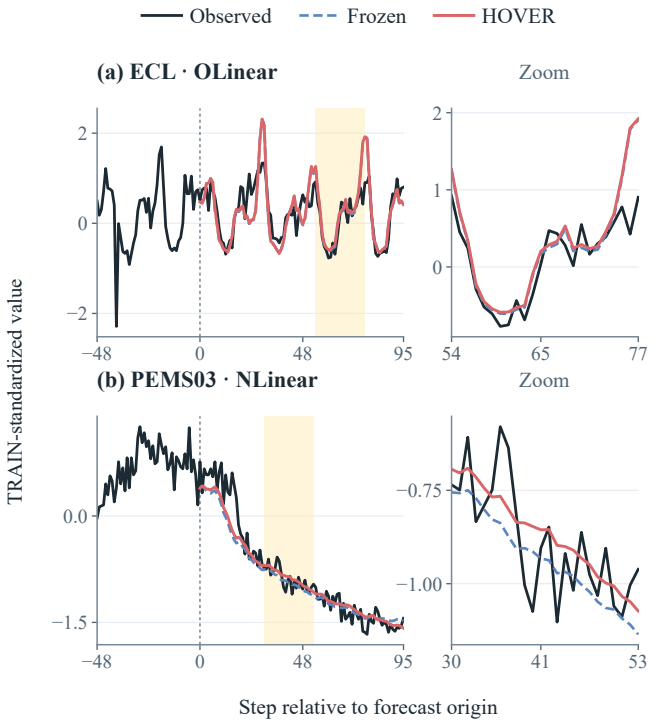


Fig. 6. Actual prediction refinement on two DEV windows: (a) frozen OLinear on ECL; (b) frozen NLinear on PEMS03. Each case shows observed history, ground truth, frozen prediction, refined prediction, and a 24-step close-up. Channel-window MSE improves by approximately 0.30% and 20.11%, respectively. Values use the training-set standardized scale.

TABLE V
PAIRED OLINEAR REFINEMENT ACROSS FORECAST HORIZONS (FIVE-SEED DEV MEANS).

Dataset	H	Frozen		HOVER		Gain (%)	
		MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	0.35435	0.38899	0.35416	0.38880	+0.05	+0.05
	192	0.47530	0.45172	0.47465	0.45196	+0.14	-0.05
	336	0.62570	0.51993	0.62278	0.51908	+0.47	+0.16
	720	0.94516	0.63596	0.94453	0.63581	+0.07	+0.02
ECL	96	0.10340	0.19160	0.10297	0.19155	+0.42	+0.02
	192	0.12006	0.20942	0.11949	0.20924	+0.48	+0.09
	336	0.13654	0.22633	0.13558	0.22550	+0.70	+0.37
	720	0.16215	0.25025	0.16112	0.25005	+0.64	+0.08
Solar	96	0.14328	0.18122	0.14003	0.18597	+2.27	-2.62
	192	0.15903	0.19355	0.15184	0.20052	+4.52	-3.60
	336	0.16661	0.19625	0.16075	0.20866	+3.52	-6.32
	720	0.15697	0.19144	0.15551	0.20659	+0.93	-7.91
PEMS03	12	0.04790	0.14493	0.04727	0.14343	+1.31	+1.03
	24	0.05222	0.15083	0.05186	0.15004	+0.68	+0.52
	48	0.05624	0.15653	0.05569	0.15542	+0.98	+0.71
	96	0.05941	0.16129	0.05888	0.16017	+0.88	+0.69

mean MSE for both backbones on ECL, Solar, and PEMS03, accounting for six of eight groups and 31 of 40 seed pairs. Mean MAE improves in five groups. Both errors rise slightly on Exchange for both backbones, and OLinear's Solar MSE gain again accompanies higher MAE. The holdout thus supports temporal persistence in several tasks while locating limits in both dataset and error measure.

The DLinear comparison extends the structural question to later observations using the same frozen bases and seeds 951–955. HOVER lowers mean TEST MSE relative to Simple by 0.36% on ECL and 2.37% on PEMS03. These results link the DEV capacity control to a separate four-method chronological comparison.

L. Direct Comparison with an Output Adapter

On DLinear, we compare with the official offline multiplicative PostY variant of δ-Adapter [10]. It multiplies the base forecast by $1 + 0.2\tanh(z)$, with z produced by its MLP on the flattened forecast. We use the same frozen checkpoints, TRAIN/DEV targets, and five seeds. PostY has about 32–36 million trainable parameters in this setting, while HOVER has 42,464.

PostY provides a comparison with an existing output correction strategy, complementing the capacity-matched Simple control. On ECL and PEMS03, HOVER lowers mean DEV MSE by 4.63% and 30.86% relative to PostY, with all five paired seeds favoring HOVER on each task. Mean MAE falls by 2.84% and 13.42%. The gains quantify the difference between these two fitted correctors on the fixed DLinear tasks and the evaluated offline PostY variant.

V. DISCUSSION

A. Complete-System Accuracy and Refinement Gains

The initial accuracy of a frozen forecaster and the increment supplied by refinement jointly determine the final system's performance. Because backbones leave residuals of different scales and structures, relative improvement and final accuracy capture different properties. Complete-system comparisons establish competitiveness; matched-backbone evaluations isolate the correction on the same checkpoint and targets. Together, they show the value of refining both predictors with substantial residuals and already accurate forecasts.

The significance of a correction also depends on the task and metric. Held-out results show that gains can persist on later observations with the configuration fixed, though not uniformly. On Solar, MSE and MAE sometimes change in opposite directions because squared and absolute errors emphasize different deviations. The error measure therefore matters when evaluating refinement.

B. Complementary Historical and Forecast-Conditioned Residuals

HOVER builds direct historical and forecast-conditioned corrections on shared historical features. History describes the observed state; the frozen prediction explicitly represents a future trajectory inferred from the same available inputs. Exposing that trajectory as a condition allows a finite corrector to use its anticipated shape without reconstructing it from historical features alone.

The paths implement distinct operations: HRP maps history directly to a residual, whereas FCRP modulates historical features using the prediction relative to the latest observation. A shared encoder provides a common representation while separate heads allow both direct and trajectory-dependent adjustments.

The capacity-matched Simple control tests the value of this organization beyond a similar number of added parameters. Retrained ablations identify more specific effects. Full HOVER shows relatively consistent gains over Only HRP, supporting forecast conditioning; removing the direct historical output, changing the fusion operation, or replacing the condition has more task-dependent effects. These results compare alternative trained structures rather than independently attributable causal effects of the jointly trained heads. Real trajectories additionally show how local corrections relate to aggregate error.

C. Refinement through Existing Forecasting Interfaces

HOVER is fitted outside the frozen predictor, preserving its variable handling, time inputs, and native forecast mapping. The same correction design can thus accommodate different backbones without reconstructing their internal representations. For fixed input and output lengths, variable sharing keeps added parameter count independent of channel number while permitting channel-specific corrections from each history and forecast.

Cross-backbone and multi-horizon experiments demonstrate separate fitting to different predictors and prediction ranges.

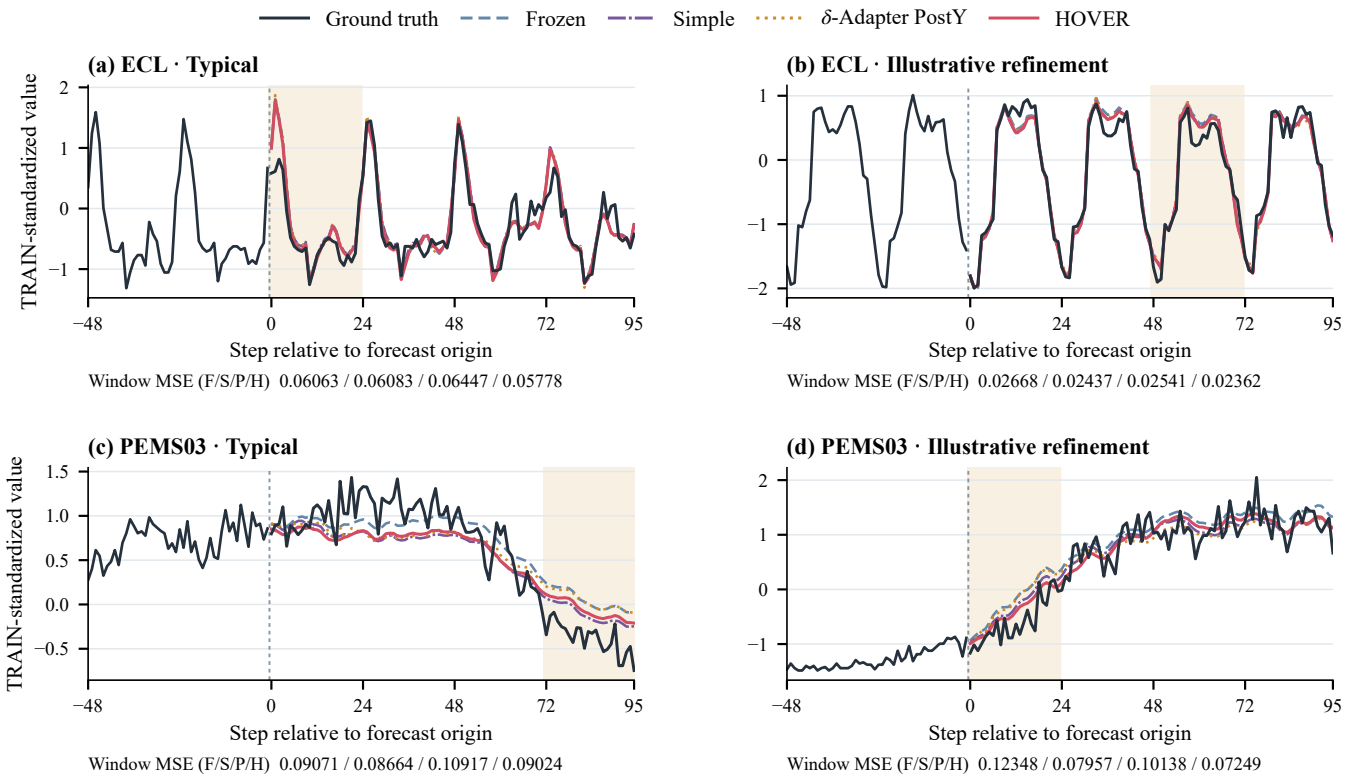


Fig. 7. Forecast trajectories under four strategies on the same frozen DLinear at matched DEV origins and channels (seed 951). Black shows the observed history and ground truth. The shaded segment has the highest frozen MSE among the four 24-step segments. Footer values give the full 96-step channel-window MSE for Frozen, Simple, PostY, and HOVER in that order.

The matched offline PostY comparison examines another residual organization: HOVER combines two paths on shared history rather than relying only on transformation of the completed forecast. Together with structural controls and actual trajectories, these findings support using the base forecast as a structured condition for instance-specific correction through an existing interface. In the example windows, the methods differ around local changes, while the paired experiments measure their net effects over full prediction horizons.

TABLE VI
CHRONOLOGICAL HELD-OUT TEST RESULTS. EACH ENTRY IS THE MEAN OVER FIVE FIXED SEEDS. GAIN IS THE RELATIVE REDUCTION IN MEAN MSE.

		MSE			MAE	
Dataset	Backbone	Frozen	HOVER	Gain	Frozen	HOVER
ECL	OLinear	0.12638	0.12580	+0.46%	0.21648	0.21633
	NLinear	0.14037	0.13211	+5.89%	0.23559	0.22656
Solar	OLinear	0.18259	0.18164	+0.52%	0.20869	0.21281
	NLinear	0.22749	0.19178	+15.70%	0.25734	0.23391
PEMS03	OLinear	0.10328	0.10285	+0.42%	0.20360	0.20278
	NLinear	0.18877	0.13561	+28.16%	0.26948	0.23154
Exchange	OLinear	0.08770	0.08775	-0.05%	0.20695	0.20745
	NLinear	0.08619	0.08645	-0.31%	0.20358	0.20410

VI. CONCLUSION

We studied refinement of trained time-series forecasters with their parameters and prediction interfaces fixed. HOVER combines recent history and the base forecast through shared historical features. Its HRP maps history directly to a correction, whereas FCRP encodes the prediction relative to the latest observation and uses low-dimensional multiplicative interaction to condition historical features. Both corrections are rescaled and added to the frozen output, with parameters shared across variables. Complete-system comparisons demonstrate competitive refined forecasts. Matched-backbone and multi-horizon experiments show incremental benefits across separately fitted predictors and horizons. Capacity-matched controls and structural ablations support the two-path organization; chronological holdout tests preserve MSE improvements in most preselected groups, with task-dependent exceptions. Output-adapter comparisons and real trajectories further characterize correction behavior. These results suggest that a trained forecast can serve not only as an output to correct but also as a structured condition expressing the backbone's learned future trajectory. Matched evidence separates correction gains from the strength of the frozen backbone. Combining that representation with recent history offers a concrete, variable-shared way to improve existing forecasts without changing the native prediction interface.

REFERENCES

[1] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[2] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640, doi: 10.24963/ijcai.2018/505.

[3] H. Zhou *et al.*, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021, pp. 11106–11115, doi: 10.1609/aaai.v35i12.17325.

[4] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 22419–22430.

[5] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?", in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 9, 2023, pp. 11121–11128, doi: 10.1609/aaai.v37i9.26317.

[6] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[7] S. Wang *et al.*, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[8] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[9] Y. Liu, H. Wu, J. Wang, and M. Long, "Non-stationary transformers: Exploring the stationarity in time series forecasting," *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 9881–9893, doi: 10.52202/068431-0718.

[10] D. Liang *et al.*, "The forecast after the forecast: A post-processing shift in time series," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[11] J. Im and H.-Y. Kwon, "COSA: Context-aware output-space adapter for test-time adaptation in time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[12] M. H. Nguyen, V. D. Do, H. H. Nguyen, D. Nguyen, K. Do, and H. Le, "Reviving error correction in modern deep time-series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 306, 2026, pp. 92824–92850.

[13] C. Chang, C.-T. Chan, W.-Y. Wang, W.-C. Peng, and T.-F. Chen, "TimeDRL: Disentangled representation learning for multivariate time-series," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2024, pp. 625–638, doi: 10.1109/ICDE60146.2024.00054.

[14] R. Zha *et al.*, "Scaling up multivariate time series pre-training with decoupled spatial-temporal representations," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2024, pp. 667–678, doi: 10.1109/ICDE60146.2024.00057.

[15] H. Miao *et al.*, "A unified replay-based continuous learning framework for spatio-temporal prediction on streaming data," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2024, pp. 1050–1062, doi: 10.1109/ICDE60146.2024.00085.

[16] T. Mu, H. Wang, C. Liang, and X. Shao, "Auto-TSF: Towards proxy-model-based meta-learning for automatic time series forecasting algorithm selection," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2025, pp. 487–500, doi: 10.1109/ICDE65448.2025.00043.

[17] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 162, 2022, pp. 27268–27286.

[18] X. Qiu, X. Wu, Y. Lin, C. Guo, J. Hu, and B. Yang, "DUET: Dual clustering enhanced multivariate time series forecasting," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD)*, 2025, pp. 1185–1196, doi: 10.1145/3690624.3709325.

[19] Z. Xu, A. Zeng, and Q. Xu, "FITS: Modeling time series with 10k parameters," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[20] S. Lin, W. Lin, W. Wu, H. Chen, and J. Yang, "SparseTSF: Modeling long-term time series forecasting with 1k parameters," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 235, 2024, pp. 30211–30226.

[21] Y. Liu *et al.*, "iTransformer: Inverted transformers are effective for time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[22] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[23] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[24] K. Yi *et al.*, "Frequency-domain MLPs are more effective learners in time series forecasting," *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 76656–76679, doi: 10.52202/075280-3349.

[25] D. Luo and X. Wang, "ModernTCN: A modern pure convolution structure for general time series analysis," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[26] S. Wang *et al.*, "TimeMixer++: A general time series pattern machine for universal predictive analysis," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[27] C. Lee, D. Lee, D. Choi, and J. Yoo, "Generalizing multi-scale time-series modeling with a single operator," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 306, 2026, pp. 64469–64502.

[28] Y. Wang *et al.*, "TimeXer: Empowering transformers for time series forecasting with exogenous variables," *Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 469–498, doi: 10.52202/079017-0015.

[29] W. Yue *et al.*, "OLinear: A linear model for time series forecasting in orthogonally transformed domain," *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025, pp. 67394–67453, doi: 10.52202/085713-2025.

[30] Y. Jiang *et al.*, "SAGDFN: A scalable adaptive graph diffusion forecasting network for multivariate time series forecasting," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2024, pp. 1255–1268, doi: 10.1109/ICDE60146.2024.00101.

[31] Y. Niu, J. Deng, L. Zhang, Z. Zhou, and Y. Tong, "Accurate and efficient multivariate time series forecasting via offline clustering," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2025, pp. 3056–3069, doi: 10.1109/ICDE65448.2025.00229.

[32] M. Wang *et al.*, "Towards lightweight time series forecasting: A patch-wise transformer with weak data enriching," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2025, doi: 10.1109/ICDE65448.2025.00100.

[33] C. Liu *et al.*, "Efficient multivariate time series forecasting via calibrated language models with privileged knowledge distillation," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2025, pp. 3165–3178, doi: 10.1109/ICDE65448.2025.00237.

[34] Q. Zhang *et al.*, "AutoHFormer: Efficient hierarchical autoregressive transformer for time series prediction," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2026, pp. 183–196, doi: 10.1109/ICDE65706.2026.00021.

[35] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 235, 2024, pp. 10148–10167.

[36] Y. Liu, H. Zhang, C. Li, X. Huang, J. Wang, and M. Long, "Timer: Generative pre-trained transformers are large time series models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 235, 2024, pp. 32369–32399.

[37] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, "Unified training of universal time series forecasting transformers," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 235, 2024, pp. 53140–53164.

[38] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, and A. Dubrawski, "MOMENT: A family of open time-series foundation models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 235, 2024, pp. 16115–16152.

[39] X. Shi *et al.*, "Time-MoE: Billion-scale time series foundation models with mixture of experts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[40] X. Qiu *et al.*, "TFB: Towards comprehensive and fair benchmarking of time series forecasting methods," *Proc. VLDB Endow.*, vol. 17, no. 9, 2024, pp. 2363–2377, doi: 10.14778/3665844.3665863.

ACKNOWLEDGMENT

OpenAI Codex assisted with portions of the English translation and with writing parts of the data-visualization code. The authors ran the plotting code, checked the figures against the experimental results, and finalized them.